FT-MUX: A Fault-Tolerant Microfluidic Multiplexer Design

Mengchu Li
TU Munich
mengchu.li@tum.de

Jiahui Peng
TU Munich
jiahui.peng@tum.de

Tsun-Ming Tseng
TU Munich
tsun-ming.tseng@tum.de

Ulf Schlichtmann
TU Munich
ulf.schlichtmann@tum.de

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133095

***Abstract*—Continuous-flow microfluidic chips are multilayered miniaturized platforms to manipulate small volumes of fluids with valves. There are two types of channels on a chip: flow channels for the reaction of fluids, and control channels for the actuation of valves. Multiplexers (MUXes) are essential microfluidic components for individually addressing many flow channels with few control channels. As the integration scale of microfluidic chips increases, the reliability of MUXes becomes a critical concern, as a single defective control channel in a MUX will affect a large part of the flow channels addressed by the MUX. This paper formally analyzes and identifies the design rules for a MUX to tolerate n defective control channels, and model the fault-tolerant MUX (FT-MUX) design problem as a binary constant weight code problem to minimize resource overheads. We demonstrate that FT-MUX improves resource efficiency by up to hundreds of times compared to the conventional fault-tolerant design method. Besides, given no less than 10 control channels, FT-MUX tolerates at least one defective control channel and addresses even more flow channels with equal or fewer resources than a standard MUX. The advantages become more significant as the integration scale increases.**

I. Introduction

Microfluidic technology enables the manufacturing of miniaturized devices, also known as microfluidic chips, that manipulate fluids at the nanoliter scale or below, exhibiting advantages in automation, rapid execution, better sensitivity, less waste, etc. In particular, the fabrication of valves using multilayer soft lithography [1] allows for the creation of microfluidic large-scale integration (mLSI) chips [2]. Such a chip consists of a flow layer (integrated) with flow channels for the storage and transportation of fluid samples and reagents, and a control layer (integrated) with control channels for the actuation of valves to manipulate the fluids. Nowadays, it is common to have hundreds to thousands of flow channels on a single small chip for high-throughput parallel applications such as cell-culture [3], cell-analysis [4], drug screening [5], serology test [6], etc. To independently address many flow channels with an affordable number of control inputs, microfluidic multiplexers (MUXes) have become indispensable for large-scale microfluidic applications [7].

A standard microfluidic multiplexer (MUX) [8] can independently address up to n flow channels with $2\lceil\log_2(n)\rceil$ control channels, as shown in Figure 1. To that end, each control channel is responsible for the pressure supply to valves on at least $\lfloor\frac{n}{2}\rfloor$ flow channels. Compared to flow channels, control channels have smaller feature sizes and are thus more prone to defects [9]. If a control channel in a standard MUX is defective, at least half of the flow channels addressed by the MUX will be affected, leading to waste of samples and reagents, contamination of products, or even disposal of the whole chip. As the integration scale of microfluidic chips increases, the reliability of MUXes becomes a critical concern [10].

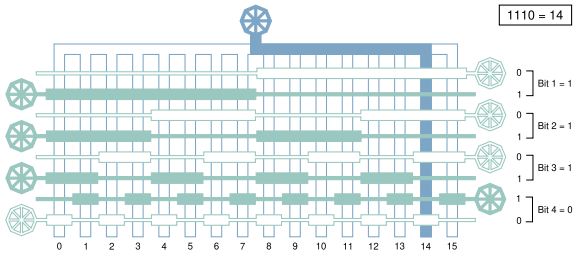

Fig. 1. A standard MUX to address 16 flow channels (colored in blue) with 8 control channels (colored in green). The pressure states in pressurized/depressurized control channels are denoted as 1 and 0, respectively. Valves are widened control channel segments. A pressurized valve blocks the flow channel underneath. Every two control channels form a pair representing one bit of a flow channel index: pressure states $(0, 1)$ and $(1, 0)$ represent bit '1' and '0', respectively. E.g., configuring the control channels to pressure states $(0, 1)$ $(0, 1)$ $(0, 1)$ $(1, 0)$ constructs a unique flow path from the fluid inlet to flow channel 14 (1110). Figure adapted from [2].

Efforts have been made in the past to improve the reliability of MUXes from two aspects:

1) Reduce the likelihood of defects. Q. Wang et al. proposed to optimize the switching order of valves to reduce the switching frequency of a MUX [11], and S. Liang et al. proposed a novel MUX design with certain valves and channels merged together [12]. However, as the number of control channels in the MUX increases, defects are almost inevitable, as reported in [12]. Since the proposed methods do not tolerate faults, once a defect happens, many flow channels will be affected, leading to significant waste.
2) Tolerate defects by introducing backups. W. Huang et al. proposed an approach to introduce redundant components considering a given set of fault scenarios [13], and Y. Zhu et al. proposed an approach to synthesize application-specific MUXes with duplicated valves and backup paths to actuate the valves [14], [15]. However, the performances of both approaches heavily depend on the given applications and are thus, in general, not predictable.

Besides, state-of-the-art methods mostly assume that defects happen at individual valves without influencing other valves

along the same control channel, which is not the case. Specifically, microfluidic chips suffer two major types of defects: blockage and leakage [16]. Blockage means a channel is disconnected such that pressure cannot pass through, and the risk of blockage inversely correlates with the feature size. Since valves are widened segments of control channels, blockage defects are more likely at the thinner parts of a control channel instead of valves. When a control channel is blocked, all valves along the channel cannot be properly pressurized. On the other hand, leakage means two control channels are unintentionally connected such that both channels share the same pressure states. When a control channel leaks to another, pressurizing/depressurizing one channel will also inevitably close/open all valves in the other channel. As conclusion, a defective valve in the MUX implies the malfunctions of all valves along a defective control channel. Thus, a fault-tolerant MUX design should target defective control channels rather than defective valves.

In this paper, we propose a rule for designing a fault-tolerant microfluidic multiplexer (FT-MUX) to tolerate n defective control channels with minimal resource overheads. We further model the FT-MUX design problem as a binary constant weight code problem, and proposes a graph model to solve it as an independent vertex set problem. We demonstrate that FT-MUX improves resource efficiency by up to hundreds of times compared to the conventional fault-tolerant design method. Besides, given no less than 10 control channels, FT-MUX tolerates at least one defective control channel and addresses even more flow channels with equal or fewer resources than a standard MUX. The performance advantages become more significant as the integration scale increases, showing that FT-MUX provides an efficient and reliable solution for fluid multiplexing in large-scale microfluidic applications.

II. Background

A. State-of-the-art MUX designs

A standard MUX consisting of n control channels can independently address up to $2^{\frac{n}{2}}$ flow channels, as shown in Figure 1, and has been widely applied in various state-of-the-art high-throughput microfluidic applications.

Recently, S. Liang et al. proposed a novel MUX design named CoMUX with a combinatorial coding strategy. A CoMUX consisting of n control channels can independently address up to $C^n_{\lceil \frac{n}{2} \rceil}$ flow channels, which is theoretically proven to be the maximum number of flow channels that a MUX can independently address [12].

B. General design rules for a microfluidic MUX

In the following, we use the CoMUX shown in Figure 2 as an example to introduce the general design rules of a MUX.

A MUX is a matrix of binary valve patterns on overlapping control and flow channels. We consider every control channel as a row of the matrix, with the topmost control channel as the first row, and every flow channel as a column of the matrix, with the leftmost flow channel as the first column. We denote the entry in the i^{th} row and j^{th} column of the matrix as '0' or '1', depending on whether there is no or one valve at the intersection of the i^{th} control channel and the j^{th} flow channel, respectively. We refer to the entries in the j^{th} column of the matrix as the **code** of the j^{th} flow channel.

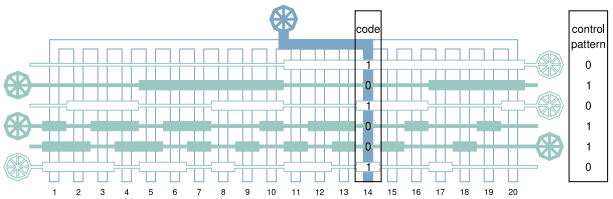

Fig. 2. A CoMUX to address 20 flow channels with 6 control channels.

Example: The code of flow channel 14 *in Figure 2 can be read as* 101001*, representing that the* 1^{st}*,* 3^{rd} *and* 6^{th} *control channels have valves on flow channel* 14*, while the* 2^{rd}*,* 4^{th} *and* 5^{th} *control channels do not have valves on flow channel* 14*.*

We denote the pressure state of a pressurized or depressurized control channel as '1' or '0', respectively, and we refer to the sequence of the pressure states of all control channels to address a flow channel f as the **control pattern** of f.

Example: The code of flow channel 14 in Figure 2 is 101001, and thus the control pattern of flow channel 14 is $\bar{1}0\bar{1}\bar{0}0\bar{1}$, i.e. 010110, such that all control channels with valves on flow channel 14 are depressurized.

To ensure that fluids can flow from a universal fluid inlet of a MUX to any flow channel without contaminating other flow channels, a MUX design must obey the following rule:

Rule 1*: The control pattern of a flow channel* f *depressurizes all valves on* f *and pressurizes at least one valve on every flow channel other than* f*.*

By negating the code of a flow channel f, the control pattern of f naturally depressurizes all valves on f. Furthermore, it is clear to see that the control pattern of a flow channel f will pressurize a valve on another flow channel f', if there is a control channel that has no valve on f, but one valve on f', i.e. there is an $i \in \mathbb{N}$, such that the i^{th} bit of the code of f is '0' and the i^{th} bit of the code of f' is '1'. S. Liang et al. further proposed and proved the following rule [12]:

Rule 2*: The maximal number of flow channels that a MUX consisting of* n *control channels can independently address is* $C^n_{\lfloor \frac{n}{2} \rfloor}$*. The codes of the flow channels can be achieved by enumerating all* n*-bit binary strings with* $\lfloor \frac{n}{2} \rfloor$ '1'*-bits.*

C. Fault model: control-channel defects in a MUX

Microfluidic chips suffer two major types of defects: *blockage* and *leakage*.

- Blockage defects happen at individual control channels. Valves on a blocked control channel cannot be pressurized and thus cannot block the fluids in the corresponding flow channels.
- Leakage defects happen between parallel control channels that are close to each other [17], [18]. Valves on a

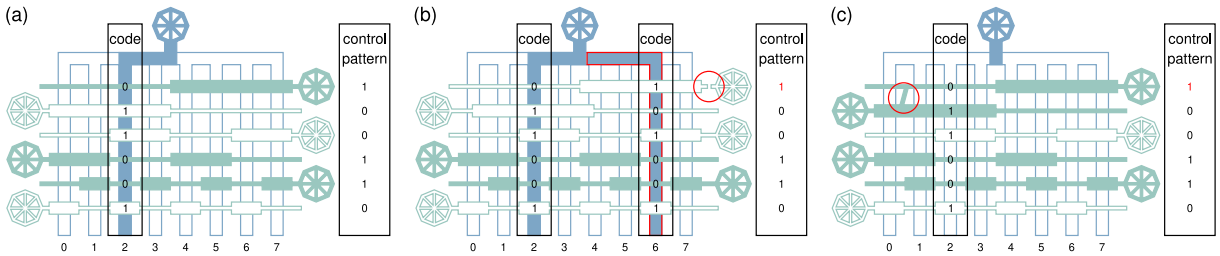

Fig. 3. (a) A standard MUX without defect. The control pattern of flow channel 2 constructs a flow path from the universal fluid inlet to flow channel 2 without contaminating other flow channels. (b) A standard MUX with a blockage defect in the 1st control channel. The control pattern of flow channel 2 contaminates flow channel 6. (c) A standard MUX with a leakage defect between the 1st and the 2nd control channels. The control pattern of flow channel 2 pressurizes the 2nd control channel unintentionally, obstructing the target flow path.

pair of leaky control channels cannot be independently pressurized but have to share the same pressure states.

For example, Figure 3 illustrates the consequences of a single blockage or leakage defect on a standard MUX.

- Suppose that the i^{th} control channel is blocked, a control pattern with a '1' at the i^{th} bit can no longer pressurize the flow channels with a '1' at the i^{th} bit of their codes. For example, the 1st control channel in the MUX in Figure 3(b) is blocked. As a result, the control pattern (100110) of flow channel 2 cannot pressurize flow channel 6 (with code 101001), leading to contamination.
- Suppose that there is a leakage defect between the i^{th} and i'^{th} control channels, a control pattern with a '1' at the i^{th} or i'^{th} bit will inevitably pressurize all flow channels with a '1' at the i^{th} or i'^{th} bit of their codes. For example, the 1st and the 2nd control channels in Figure 3(c) leak with each other. As a result, the control pattern (100110) of flow channel 2 pressurizes the valve at the intersection between flow channel 2 and the 2nd control channel, obstructing the flow path from the universal fluid inlet to flow channel 2.

In general, by studying the working mechanism of state-of-the-art MUXes, we conclude that even under the single-fault assumption, depending on the type and position of the defect, a defective control channel in a standard MUX will affect $50\%-100\%$ of the flow channels addressed by the MUX, and a defective control channel in a CoMUX will affect $33\%-50\%$ of the flow channels addressed by the MUX.

Flow channels affected by the defects become unavailable or at risk of contamination, which leads to the waste of reagents, samples, and chip area. Moreover, the single-fault assumption does not always hold in practice. The risk of defects increases with the integration scale of the chip, and more damage is expected if there are more defective control channels in the MUX. Thus, the design of a fault-tolerant MUX with minimal resource overhead is essential.

III. Problem Formulation

Fault scenario:

- Blockage can happen to any individual control channel.
- Leakage can happen between any two control channels.

Input:

- The number of control channels in a MUX, denoted as n.
- The number of control channel defects that should be tolerated by a MUX, denoted as k.

Output:

- A fault-tolerant MUX design consisting of n control channels to independently address every flow channel in the MUX regardless of k control channel defects.

Objective:

- Maximize the number of flow channels addressed by the fault-tolerant MUX.

IV. Method

A. Designs rules for a fault-tolerant MUX

For convenience, we denote the code of a flow channel f as c_f. As mentioned in Section II-B, a MUX design must obey Rule 1, i.e. the control pattern of a flow channel f depressurizes all valves on f and pressurizes at least one valve on every flow channel other than f. To develop the design rule for a fault-tolerant MUX, we first transform Rule 1 to the following equivalent statement:

Rule 1^**: For any two flow channels* f *and* f'*,* c_f *and* c'_f *must differ by at least two bits, among which there is at least one bit at which* c_f *takes* '1' *and* c'_f *takes* '0' *and one bit at which* c_f *takes* '0' *and* c'_f *takes* '1'*.*

Next, we propose the following rule for designing a MUX that tolerates one blockage or leakage defect:

Rule 3*: For any two flow channels* f *and* f'*,* c_f *and* c'_f *must differ by at least four bits, among which there are at least two bits at which* c_f *takes* '1' *and* c'_f *takes* '0' *and two bits at which* c_f *takes* '0' *and* c'_f *takes* '1'*.*

In the following, we prove with illustrations that if a MUX satisfies Rule 3, as shown in Figure 4(a), it satisfies Rule 1^* even when there is a blockage or leakage defect.

- As shown in Figure 4(b), if a blockage defect happens in a control channel, valves on that channel are depressurized, and the corresponding bits in c_f and $c_{f'}$ should be regarded as '0'. In that case, at least three bits of c_f and $c_{f'}$ remain different, and Rule 1^* is satisfied.
- As shown in Figure 4(c), if a leakage defect happens between two control channels, one of which has a valve

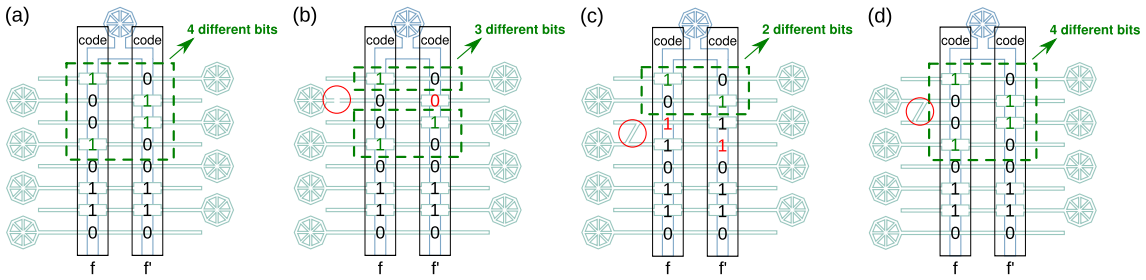

Fig. 4. (a) A design that satisfies Rule 3. (b) Suppose that the 2nd control channel is blocked, valves along the channel cannot be pressurized and should be ignored, i.e., the 2nd bits of c_f and $c_{f'}$ should be regarded as 0. Still, three bits of c_f and c'_f remain different, among which at least one bit of both c_f and c'_f takes '1'. (c) Suppose that the 3rd and the 4th control channels leak with each other, pressurizing one channel will inevitably pressurize the other. Since the control pattern of f requires the 4th control channel to be depressurized, the 3rd control channel must also be depressurized to ensure that f is not obstructed. Thus, the 3rd bit of c_f should be regarded as '1'. Similarly, the 4th bit of $c_{f'}$ should be regarded as '1'. Still, the first two bits of c_f and c'_f remain different, among which one bit of both c_f and c'_f takes '1'. (d) Suppose that the 2nd and the 3rd control channels leak with each other. Since the 2nd and the 3rd bits of c_f are both '0', i.e., the pressure states in the 2nd and the 3rd control channels are the same in the control pattern of f, the leakage does not affect c_f. Similarly, since the 2nd and the 3rd bits of $c_{f'}$ are both '1', the leakage does not affect $c_{f'}$.

on f (f') while the other has no valve on f (f'), both control channels must be depressurized in the control pattern of f (f'), and the corresponding bits in c_f ($c_{f'}$) should be regarded as '1'. In that case, at least two bits of c_f and $c_{f'}$ remain different, and Rule 1^* is satisfied.

- As shown in Figure 4(d), if a leakage defect happens between two control channels, both or neither of which have a valve on f (f'), c_f ($c_{f'}$) does not change and Rule 1^* is satisfied. □

We further generalize Rule 3 into Rule 4 for designing a MUX that tolerates $k \in \mathbb{N}$ blockage or leakage defects:

Rule 4: For any two flow channels f and f', c_f and c'_f must differ by at least $2(k+1)$ bits, among which there are at least $k+1$ bits at which c_f takes '1' and c'_f takes '0' and $k+1$ bits at which c_f takes '0' and c'_f takes '1'.

A proof can be derived using the same strategy as above.

B. Binary constant weight code model

To maximize the number of flow channels addressed by the fault-tolerant MUX, we model the fault-tolerant MUX design problem as a binary constant weight code problem. Specifically, a binary constant weight code with parameters n, w, d is a set $\mathcal{C}$ of binary words of length n and Hamming weight w such that the Hamming distance between any two binary words in $\mathcal{C}$ is at least d [19]. Specifically, a binary word of length n is a string of n bits; the Hamming weight of a binary word is the number of '1'-bits in the binary word; and the Hamming distance between two binary words of equal length is the number of bits at which the two words take different values.

According to Rule 2, the largest set of codes of flow channels in a MUX with n control channels, denoted as $\mathcal{M}_n$, can be achieved by enumerating all n-bit binary strings with $\lfloor \frac{n}{2} \rfloor$ '1'-bits. Since a fault-tolerant MUX is a MUX with extra constraints on the available codes, the codes of the flow channels in a fault-tolerant MUX with n control channels should be a subset of $\mathcal{M}_n$. Thus, we derive the following statement:

To address most flow channels with a fault-tolerant MUX consisting of n control channels, the code of every flow channel should be a binary word of length n and Hamming weight $\lfloor \frac{n}{2} \rfloor$.

We observed that for two binary words w_1 and w_2 of the same length and weight, if their Hamming distance is $2k$, there must be exactly k bits at which w_1 takes '1' and w_2 takes '0' and k bits at which w_1 takes '0' and w_2 takes '1'. Thus, given the same length and Hamming weight of the codes of all flow channels in a MUX, Rule 4 can be simplified as:

Rule 4^*: For any two flow channels f and f', the Hamming distance between c_f and c'_f must be larger than or equal to $2(k+1)$.

Therefore, the problem described in Section III can be modeled as the following binary constant weight code problem:

Finding a largest possible binary constant weight code with length n, weight $\lfloor \frac{n}{2} \rfloor$, and Hamming distance $\geq 2(k+1)$.

The binary constant weight code problem has been extensively studied in the field of information theory [20], [21]. Some upper bounds and lower bounds of the maximum size of a binary constant weight code can be computed [22], but it is generally difficult to find the largest possible code for most parameters. To approximate the theoretical upper bounds, we propose a graph model and transform the binary constant weight code problem into an independent vertex set problem.

Given n control channels and k to-be-tolerated defects, we build a graph $G = (V, E)$, where each vertex $v \in V$ represents a binary word of length n and Hamming weight $\lfloor \frac{n}{2} \rfloor$. If the Hamming distance between the binary words represented by any two vertices v_1 and v_2 is smaller than $2(k+1)$, we add an edge $(v_1, v_2) \in E$. Therefore, any independent vertex set of the graph represents a binary constant weight code with length n, weight $\lfloor \frac{n}{2} \rfloor$, and Hamming distance $\geq 2(k+1)$. The problem of finding a largest possible binary constant weight code is thus equivalent to finding a maximum independent vertex set in G.

The maximum independent vertex set problem is a classic

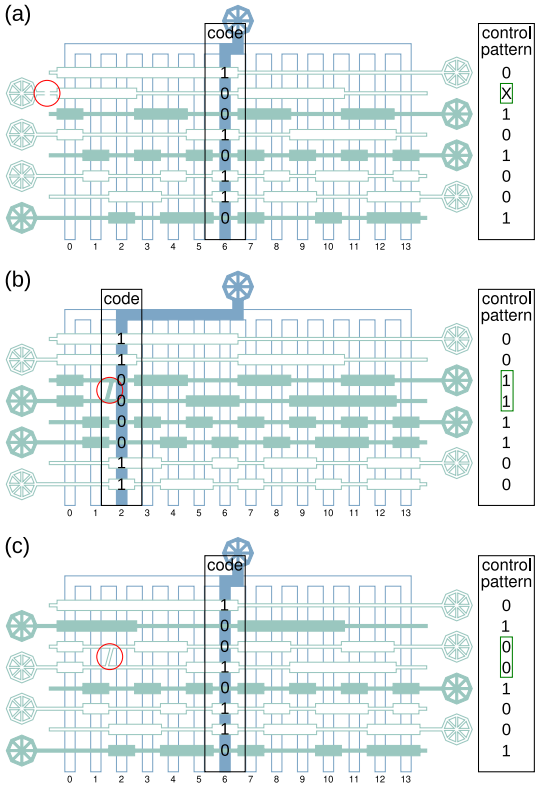

Fig. 5. (a) Control pattern in case of a blockage defect. (b)(c) Control patterns in case of a leakage defect.

NP-hard problem, for which there are numerous studies and algorithms. In this work, we solve the problem with a simple mixed integer linear programming model:

For every vertex $v \in V$, we introduce a binary variable b_v to represent whether v is in the independent vertex set. We then introduce the following constraint for each edge $(v_1, v_2) \in E$:

$$b_{v_1} + b_{v_2} \leq 1,$$

and set the objective function as:

$$\textit{Maximize:} \sum_{v \in V} b_v.$$

C. Control patterns of a fault-tolerant MUX

Based on a binary constant weight code $\mathcal{C}$ with length n, weight $\lfloor \frac{n}{2} \rfloor$, and Hamming distance $2(k+1)$, we can design a MUX that addresses $|\mathcal{C}|$ flow channels with n control channels tolerating k defects, where $|\mathcal{C}|$ denotes the cardinality of set $\mathcal{C}$.

For example, Figure 5 shows a MUX designed based on the largest binary constant weight code with length 8, weight 4, and Hamming distance 4, which addresses 14 flow channels with 8 control channels tolerating one defect. The control pattern to address an arbitrary flow channel with the fault-tolerant MUX depends on the code of the flow channel and the type of defect.

- If there is a blockage defect on any control channel, the control pattern to address a flow channel remains the negation of the code of that flow channel. E.g. the 2nd control channel of the fault-tolerant MUX in Figure 5(a) suffers a blockage defect. Nevertheless, the control pattern to address flow channel 6 with code 10010110 remains $\bar{1}\bar{0}\bar{0}\bar{1}\bar{0}\bar{1}\bar{1}\bar{0}$, i.e. 01101001. In fact, since the second control channel cannot be properly pressurized, we can also denote the second bit of the control pattern as 'X', representing a don't care term.
- If there is a leakage defect between the i^{th} and i'^{th} control channels, e.g. between the 3rd and the 4th control channels, as shown in Figure 5(b) and (c), there are two cases:
  - If the i^{th} and i'^{th} bits of the code of the to-be-addressed flow channel take the same value, i.e. 00 or 11, the control pattern remains the negation of the code of that channel. E.g. as shown in Figure 5(b), the control pattern to address flow channel 2 with code 11000011 remains $\bar{1}\bar{1}\bar{0}\bar{0}\bar{0}\bar{0}\bar{1}\bar{1}$, i.e. 00111100, since the 3rd and the 4th bits of the code of channel 2 are both '0'.
  - If the i^{th} and i'^{th} bits of the code of the to-be-addressed flow channel take different values, i.e. 01 or 10, we change the i^{th} and i'^{th} bits of the control pattern to 00, and keep other bits of the control pattern the negation of the code of that channel. E.g. as shown in Figure 5(c), the control pattern to address flow channel 6 with code 10010110 becomes $\bar{1}\bar{0}\bar{0}\bar{0}\bar{1}\bar{1}\bar{0}$, i.e. 01001001, since the 3rd and the 4th bits of the code of channel 6 are '0' and '1', respectively.

V. Results

We compare our fault-tolerant MUX design, abbreviated as FT-MUX, with two MUX designs: the mainstream design, referred to as the standard MUX [8] and a novel design, referred to as CoMUX [12]. Since the standard MUX and CoMUX do not tolerate defects, we apply the conventional fault-tolerant design method [13] to duplicate the control channels and valves in these MUXes. Specifically, since it is in general not possible to predict the position of defects, to tolerate one or two defects, all control channels in a standard MUX/CoMUX need to be doubled or tripled, respectively. We denote MUXes that tolerate one or two defects by adding (I) or (II) next to their names, respectively.

To achieve the optimal code for our FT-MUX, we run the mixed integer linear programming model on a computer with two Xeon Gold 6126 2.6 GHz processors. We achieved the optimal solutions, i.e. the largest possible codes, for FT-MUX(I) and FT-MUX(II) with up to 12 control channels within a few seconds. Given more than 12 control channels, the model cannot guarantee the optimality of the solution. In those cases, we adopt the constant weight code table published in [23]. The results of the comparison are shown in Figure 6. Based on the comparison, we make the following conclusions:

- FT-MUX greatly improved the resource efficiency compared to the conventional fault-tolerant design method.

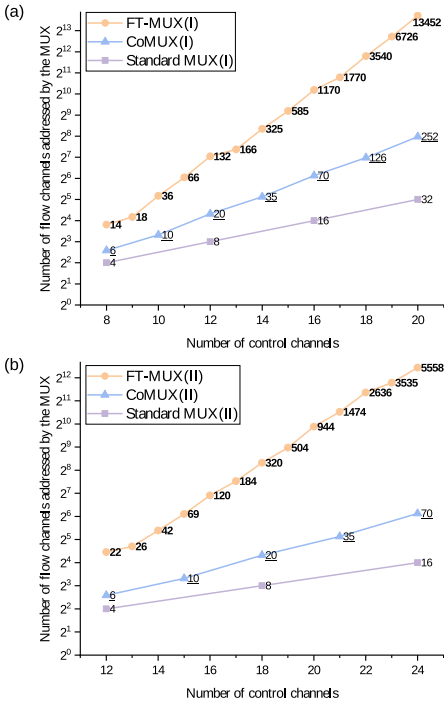

Fig. 6. Comparison among (a) MUXes that tolerate one defect; (b) MUXes that tolerate two defects.

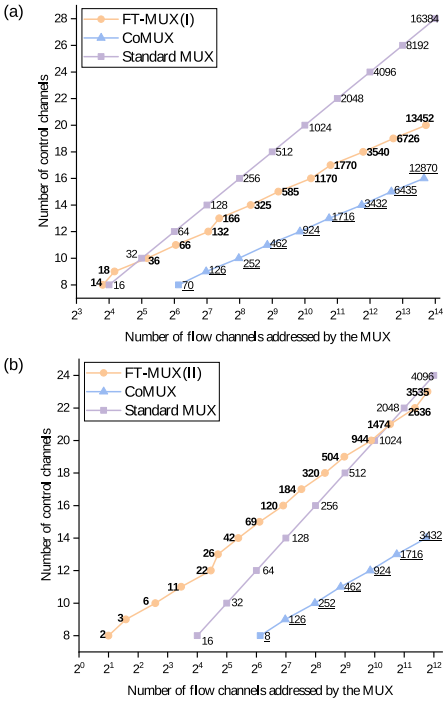

Fig. 7. Comparison among state-of-the-art MUXes that tolerate no defect and (a) FT-MUXes that tolerate one defect; (b) FT-MUXes that tolerate two defects.

In particular, the efficiency advantages increase with the integration scale. Given 12 control channels, FT-MUX(I) addresses 6 and 16 times more flow channels than CoMUX(I) and standard MUX(I), respectively; and given 20 control channels, FT-MUX(I) addresses 53 and 420 times more flow channels than CoMUX(I) and standard MUX(I), respectively.

- FT-MUX is the only reasonable MUX design to tolerate more than one defect. In case two defects need to be tolerated, with up to 18 control channels, CoMUX(II) and standard MUX(II) require a similar or even larger number of control channels than the to-be-addressed flow channels, making the multiplexing meaningless. In the meanwhile, FT-MUX(II) can still reliably address many flow channels with only a few control channels.

Besides, given 24 control channels, FT-MUX(III) can tolerate three defects while addressing 2576 flow channels; and given 32 control channels, FT-MUX(IV) can tolerate four defects while addressing 3038 flow channels.

To evaluate the resource usage for implementing the fault-tolerant features, we also compare FT-MUXes that tolerate one or two defects to the standard MUXes and CoMUXes without fault-tolerant features. Figure 7 shows the results of the comparison.

- Comparison to standard MUXes. Given no less than 10 control channels, FT-MUX(I) tolerates one defect while addressing more flow channels than a standard MUX that tolerates no defect; and given no less than 22 control channels, FT-MUX(II) tolerates two defects while addressing more flow channels than a standard MUX that tolerates no defect. In other words, FT-MUX significantly outperforms the standard MUX, not only in reliability but also in resource efficiency.
- Comparison to CoMUXes. In general, to address a similar number of flow channels, FT-MUXes that tolerate one defect use 3 to 4 more control channels than CoMUXes that tolerate no defect; and FT-MUXes that tolerate two defects use 7 to 9 more flow channels than CoMUXes that tolerate no defect. It is worth noticing that the resource for implementing the fault-tolerant features does not increase significantly with the number of to-be-addressed flow channels, demonstrating good scalability of FT-MUXes.

VI. Conclusion

Fluid multiplexing is one of the most important operations on microfluidic chips. The reliability of fluid multiplexers, or MUXes, has thus been drawing more and more attention in recent years. This paper proposes the design rules for a fault-tolerant MUX (FT-MUX) for the first time and models the FT-MUX design problem as a constant weight code problem to minimize resource overheads. We demonstrated that FT-MUX significantly outperforms the state-of-the-art MUX designs and fault-tolerant design methods. As the integration scale of microfluidic chips increases, FT-MUX enables an efficient and reliable solution to execute microfluidic applications.

Acknowledgment

This work is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation, No. 515003344).

REFERENCES

[1] Marc A Unger, Hou-Pu Chou, Todd Thorsen, Axel Scherer, and Stephen R Quake. Monolithic microfabricated valves and pumps by multilayer soft lithography. *science*, 288(5463):113–116, 2000.

[2] Jessica Melin and Stephen R Quake. Microfluidic large-scale integration: the evolution of design rules for biological automation. *Annu. Rev. Biophys. Biomol. Struct.*, 36:213–231, 2007.

[3] Nina Compera, Scott Atwell, Johannes Wirth, Christine von Törne, Stefanie M Hauck, and Matthias Meier. Adipose microtissue-on-chip: a 3d cell culture platform for differentiation, stimulation, and proteomic analysis of human adipocytes. *Lab on a Chip*, 22(17):3172–3186, 2022.

[4] Mohamed Ibrahim, Krishnendu Chakrabarty, and Ulf Schlichtmann. Cosyn: Efficient single-cell analysis using a hybrid microfluidic platform. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 1673–1678. IEEE, 2017.

[5] Brooke Schuster, Michael Junkin, Sara Saheb Kashaf, Isabel Romero-Calvo, Kori Kirby, Jonathan Matthews, Christopher R Weber, Andrey Rzhetsky, Kevin P White, and Savaş Tay. Automated microfluidic platform for dynamic and combinatorial drug screening of tumor organoids. *Nature communications*, 11(1):5271, 2020.

[6] Roberto Rodriguez-Moncayo, Diana F Cedillo-Alcantar, Pablo E Guevara-Pantoja, Oriana G Chavez-Pineda, Jose A Hernandez-Ortiz, Josue U Amador-Hernandez, Gustavo Rojas-Velasco, Fausto Sanchez-Muñoz, Daniel Manzur-Sandoval, Luis D Patino-Lopez, et al. A high-throughput multiplexed microfluidic device for covid-19 serology assays. *Lab on a Chip*, 21(1):93–104, 2021.

[7] Tsun-Ming Tseng, Mengchu Li, Daniel Nestor Freitas, Amy Mongersun, Ismail Emre Araci, Tsung-Yi Ho, and Ulf Schlichtmann. Columba s: A scalable co-layout design automation tool for microfluidic large-scale integration. In *Proceedings of the 55th Annual Design Automation Conference*, pages 1–6, 2018.

[8] Todd Thorsen, Sebastian J Maerkl, and Stephen R Quake. Microfluidic large-scale integration. *Science*, 298(5593):580–584, 2002.

[9] Mengchu Li, Yushen Zhang, Ju Young Lee, Hudson Gasvoda, Ismail Emre Araci, Tsun-Ming Tseng, and Ulf Schlichtmann. Integrated test module design for microfluidic large-scale integration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.

[10] Siyuan Liang, Meng Lian, Mengchu Li, Tsun-Ming Tseng, Ulf Schlichtmann, and Tsung-Yi Ho. Armm: Adaptive reliability quantification model of microfluidic designs and its graph-transformer-based implementation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2023.

[11] Qin Wang, Shiliang Zuo, Hailong Yao, Tsung-Yi Ho, Bing Li, Ulf Schlichtmann, and Yici Cai. Hamming-distance-based valve-switching optimization for control-layer multiplexing in flow-based microfluidic biochips. In *2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 524–529. IEEE, 2017.

[12] Siyuan Liang, Mengchu Li, Tsun-Ming Tseng, Ulf Schlichtmann, and Tsung-Yi Ho. Comux: Combinatorial-coding-based high-performance microfluidic control multiplexer design. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, pages 1–9, 2022.

[13] Wei-Lun Huang, Ankur Gupta, Sudip Roy, Tsung-Yi Ho, and Paul Pop. Fast architecture-level synthesis of fault-tolerant flow-based microfluidic biochips. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 1667–1672. IEEE, 2017.

[14] Ying Zhu, Bing Li, Tsung-Yi Ho, Qin Wang, Hailong Yao, Robert Wille, and UIf Schlichtmann. Multi-channel and fault-tolerant control multiplexing for flow-based microfluidic biochips. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2018.

[15] Ying Zhu, Xing Huang, Bing Li, Tsung-Yi Ho, Qin Wang, Hailong Yao, Robert Wille, and Ulf Schlichtmann. Multicontrol: Advanced control-logic synthesis for flow-based microfluidic biochips. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(10):2489–2502, 2019.

[16] Kai Hu, Feiqiao Yu, Tsung-Yi Ho, and Krishnendu Chakrabarty. Testing of flow-based microfluidic biochips: Fault modeling, test generation, and experimental demonstration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(10):1463–1475, 2014.

[17] Jiaxing Wang, Mingxin Zheng, Wei Wang, and Zhihong Li. Optimal protocol for moulding pdms with a pdms master. *Chips & Tips (Lab on a Chip)*, 6, 2010.

[18] Mengchu Li, Hanchen Gu, Yushen Zhang, Siyuan Liang, Hudson Gasvoda, Rana Altay, Ismail Emre Araci, Tsun-Ming Tsneg, Tsung-Yi Ho, and Ulf Schlichtmann. Late breaking results: Efficient built-in self-test for microfluidic large-scale integration (mlsi). In *Proceedings of the 62th Annual Design Automation Conference*, pages 1–6, 2024.

[19] Kenneth S Suslick. Encyclopedia of physical science and technology. *Sonoluminescence and sonochemistry, 3rd edn. Elsevier Science Ltd, Massachusetts*, pages 1–20, 2001.

[20] A. E. Brouwer, J. B. Shearer, N. J. A. Sloane, and W. D. Smith. A new table of constant weight codes. *IEEE Trans. Inf. Theory*, 36:1334–1380, 1990.

[21] Derek H Smith, Lesley A Hughes, and Stephanie Perkins. A new table of constant weight codes of length greater than 28. *the electronic journal of combinatorics*, 13(1):A2, 2006.

[22] Selmer Johnson. A new upper bound for error-correcting codes. *IRE Transactions on Information Theory*, 8(3):203–207, 1962.

[23] Bounds for binary constant weight codes. https://www.win.tue.nl/ aeb/codes/Andw.html. Accessed: 2024-07-05.